

ASSIGNMENT 6

Priyanshu Dey_703

2026-03-19

PROBLEM SHEET 6.

Application of Subset Selection Methods

Consider the “Hitters” dataset. We need the ISLR library to load the data. Our goal is to predict a baseball player’s salary based on the available predictors.

```
library(ISLR)
library(leaps)
library(glmnet)

baseball_data = Hitters
head(baseball_data)
```

##	AtBat	Hits	HmRun	Runs	RBI	Walks	Years	CAAtBat	CHits
## CHmRun									
## -Andy Allanson	293	66	1	30	29	14	1	293	66
1									
## -Alan Ashby	315	81	7	24	38	39	14	3449	835
69									
## -Alvin Davis	479	130	18	66	72	76	3	1624	457
63									
## -Andre Dawson	496	141	20	65	78	37	11	5628	1575
225									
## -Andres Galarraga	321	87	10	39	42	30	2	396	101
12									
## -Alfredo Griffin	594	169	4	74	51	35	11	4408	1133
19									
##	CRuns	CRBI	CWalks	League	Division	PutOuts	Assists	Errors	
## -Andy Allanson	30	29	14	A	E	446	33	20	
## -Alan Ashby	321	414	375	N	W	632	43	10	
## -Alvin Davis	224	266	263	A	W	880	82	14	
## -Andre Dawson	828	838	354	N	E	200	11	3	
## -Andres Galarraga	48	46	33	N	E	805	40	4	
## -Alfredo Griffin	501	336	194	A	W	282	421	25	
##	Salary	NewLeague							
## -Andy Allanson	NA	A							
## -Alan Ashby	475.0	N							
## -Alvin Davis	480.0	A							
## -Andre Dawson	500.0	N							
## -Andres Galarraga	91.5	N							
## -Alfredo Griffin	750.0	A							

1. Clean the data by removing any rows that have missing values.

```
clean_hitters = na.omit(baseball_data)
```

2. Split the data randomly, keeping 80% for training and the rest for testing.

```
set.seed(123)
total_rows = nrow(clean_hitters)
train_idx = sample(1:total_rows, 210)
train_data = clean_hitters[train_idx, ]
test_data = clean_hitters[-train_idx, ]
```

3. Perform best subset selection on the training data. We'll check models with up to 19 variables and compare them using Adjusted R-squared, BIC, and Cp.

```
subset_model = regsubsets(Salary ~ ., data = train_data, nvmax = 19)
model_summary = summary(subset_model)
```

```
best_r2 = which.max(model_summary$adjr2)
best_bic = which.min(model_summary$bic)
best_cp = which.min(model_summary$cp)
```

```
cat("Best model coefficients under BIC:\n")
```

```
## Best model coefficients under BIC:
```

```
print(coef(subset_model, best_bic))
```

```
## (Intercept)      Walks      CAtBat      CHits      CHmRun
DivisionW
## 149.5853608      5.9582020     -0.5241611      2.0687418      1.7652811 -
129.0165012
```

```
cat("\nBest model coefficients under Adjusted R^2:\n")
```

```
##
```

```
## Best model coefficients under Adjusted R^2:
```

```
print(coef(subset_model, best_r2))
```

```
## (Intercept)      AtBat      Hits      Walks      CAtBat
CRuns
## 154.1594365     -1.8722264      5.7547106      6.3852096     -0.2092154
1.7712871
##          CRBI      CWalks      DivisionW      PutOuts      Assists
Errors
##  1.1419024     -0.8989348    -121.8633686      0.2013298      0.5363804  -
6.4726568
##   NewLeagueN
##    75.8378982
```

```
cat("\nBest model coefficients under Cp:\n")
```

```
##
## Best model coefficients under Cp:

print(coef(subset_model, best_cp))

## (Intercept)          AtBat          Hits          Walks          CAtBat
CRuns
## 134.1593585   -1.9328699    5.8271814    6.5037245   -0.2063323
1.7820180
##          CRBI          CWalks    DivisionW          PutOuts          Assists
NewLeagueN
## 1.1120229   -0.8879140 -121.2135304    0.1925628    0.3316973
71.1433129
```

4. Find the training MSE and test MSE for each of the best models selected above.

```
predict.regsbsets = function(object, newdata, id, ...){
  form = as.formula(object$call[[2]])
  mat = model.matrix(form, newdata)
  coefi = coef(object, id=id)
  xvars = names(coefi)
  mat[, xvars] %*% coefi
}

# MSE for the best BIC Model
train_pred_b = predict.regsbsets(subset_model, train_data, best_bic)
test_pred_b = predict.regsbsets(subset_model, test_data, best_bic)
train_mse_bic = mean((train_data$Salary - train_pred_b)^2)
test_mse_bic = mean((test_data$Salary - test_pred_b)^2)

# MSE for the best Adjusted R^2 Model
train_pred_r = predict.regsbsets(subset_model, train_data, best_r2)
test_pred_r = predict.regsbsets(subset_model, test_data, best_r2)
train_mse_r2 = mean((train_data$Salary - train_pred_r)^2)
test_mse_r2 = mean((test_data$Salary - test_pred_r)^2)

# MSE for the best Cp Model
train_pred_c = predict.regsbsets(subset_model, train_data, best_cp)
test_pred_c = predict.regsbsets(subset_model, test_data, best_cp)
train_mse_cp = mean((train_data$Salary - train_pred_c)^2)
test_mse_cp = mean((test_data$Salary - test_pred_c)^2)

cat("For BIC model,\nTraining MSE is", train_mse_bic, "\nTest MSE is",
test_mse_bic, "\n\n")

## For BIC model,
## Training MSE is 97026.43
## Test MSE is 153090.5

cat("For Adjusted R^2 model,\nTraining MSE is", train_mse_r2, "\nTest MSE
is", test_mse_r2, "\n\n")
```

```
## For Adjusted R^2 model,
## Training MSE is 88935.8
## Test MSE is 130066.9

cat("For Cp model,\nTraining MSE is", train_mse_cp, "\nTest MSE is",
test_mse_cp, "\n\n")

## For Cp model,
## Training MSE is 89790.63
## Test MSE is 125652.3
```

5. Repeat the process using the forward stepwise selection method.

```
fwd_model = regsubsets(Salary ~ ., data = train_data, nvmax = 19, method =
"forward")
fwd_summary = summary(fwd_model)

fwd_r2_best = which.max(fwd_summary$adjr2)
fwd_bic_best = which.min(fwd_summary$bic)
fwd_cp_best = which.min(fwd_summary$cp)

cat("Best forward model coefficients under BIC:\n")

## Best forward model coefficients under BIC:

print(coef(fwd_model, fwd_bic_best))

## (Intercept)      Walks      CAtBat      CHits      CRuns
## 127.8159745      6.4746441     -0.4127407      1.1133040      1.0341530
## 0.8520825
##      CWalks      DivisionW
## -0.5432536 -127.3877839

cat("\nBest forward model coefficients under Adjusted R^2:\n")

##
## Best forward model coefficients under Adjusted R^2:

print(coef(fwd_model, fwd_r2_best))

## (Intercept)      AtBat      Hits      Walks      Years
## 207.8865937     -1.6603271      4.9552022      6.0893338     -12.4996617
## 0.2504738
##      CHits      CRuns      CRBI      CWalks      DivisionW
## 0.3954684      1.4147636      1.0607231     -0.7225505     -123.4055931
## 0.1923965
##      Assists      Errors      NewLeagueN
## 0.5131110     -7.0115407      70.8774554
```

```

cat("\nBest forward model coefficients under Cp:\n")

##
## Best forward model coefficients under Cp:

print(coef(fwd_model, fwd_cp_best))

## (Intercept)      Walks      CAtBat      CHits      CRuns
CRBI
## 110.1015291    5.2753942   -0.4498891    1.0510348    1.3425171
0.9381944
##      CWalks    DivisionW      PutOuts      Assists      Errors
NewLeagueN
##  -0.5426157 -125.5159872    0.1609710    0.4662319   -7.6994411
74.6774479

# Calculating Test MSE for the best Adjusted R^2 forward model
fwd_train_preds = predict.regsbsets(fwd_model, train_data, fwd_r2_best)
fwd_test_preds = predict.regsbsets(fwd_model, test_data, fwd_r2_best)
fwd_train_mse = mean((train_data$Salary - fwd_train_preds)^2)
fwd_test_mse = mean((test_data$Salary - fwd_test_preds)^2)

cat("\nFor Forward Selection (Adjusted R^2),\nTraining MSE is",
fwd_train_mse, "\nTest MSE is", fwd_test_mse, "\n")

##
## For Forward Selection (Adjusted R^2),
## Training MSE is 88287.08
## Test MSE is 134001.7

```

6. Repeat using the backward stepwise selection method.

```

bwd_model = regsubsets(Salary ~ ., data = train_data, nvmax = 19, method =
"backward")
bwd_summary = summary(bwd_model)

bwd_r2_best = which.max(bwd_summary$adjr2)
bwd_bic_best = which.min(bwd_summary$bic)
bwd_cp_best = which.min(bwd_summary$cp)

cat("Best backward model coefficients under BIC:\n")

## Best backward model coefficients under BIC:

print(coef(bwd_model, bwd_bic_best))

## (Intercept)      Walks      CAtBat      CHits      CHmRun
DivisionW
## 149.5853608    5.9582020   -0.5241611    2.0687418    1.7652811 -
129.0165012

cat("\nBest backward model coefficients under Adjusted R^2:\n")

```

```
##
## Best backward model coefficients under Adjusted R^2:

print(coef(bwd_model, bwd_r2_best))

## (Intercept)          AtBat          Hits          Walks          CAtBat
CHits
## 151.3709120 -1.5393497  4.6939494  6.1305798 -0.3519984
1.1424820
## CHmRun          CRuns          CWalks          DivisionW          PutOuts
Assists
## 2.3082206 0.9269527 -0.5968973 -120.5643234 0.1975004
0.5618678
## Errors          NewLeagueN
## -7.1940771 71.8060203

cat("\nBest backward model coefficients under Cp:\n")

##
## Best backward model coefficients under Cp:

print(coef(bwd_model, bwd_cp_best))

## (Intercept)          AtBat          Hits          Walks          CAtBat
CHits
## 151.3709120 -1.5393497  4.6939494  6.1305798 -0.3519984
1.1424820
## CHmRun          CRuns          CWalks          DivisionW          PutOuts
Assists
## 2.3082206 0.9269527 -0.5968973 -120.5643234 0.1975004
0.5618678
## Errors          NewLeagueN
## -7.1940771 71.8060203

# Calculating Test MSE for the best Adjusted R^2 backward model
bwd_train_preds = predict.regsbsets(bwd_model, train_data, bwd_r2_best)
bwd_test_preds = predict.regsbsets(bwd_model, test_data, bwd_r2_best)
bwd_train_mse = mean((train_data$Salary - bwd_train_preds)^2)
bwd_test_mse = mean((test_data$Salary - bwd_test_preds)^2)

cat("\nFor Backward Selection (Adjusted R^2),\nTraining MSE is",
bwd_train_mse, "\nTest MSE is", bwd_test_mse, "\n")

##
## For Backward Selection (Adjusted R^2),
## Training MSE is 88865.02
## Test MSE is 134736.7
```

7. Compare the results of the three methods.

Best Subset Selection checks every possible combination to find the best model, but it takes a lot of computing power. Forward Selection adds predictors one by one, while Backward Selection starts with everything and drops predictors step by step.

Looking at the metrics, the Cp criterion usually picked very simple models with high MSE, which means they were underfitting the data. The models picked using Adjusted R-squared kept more predictors and had much better accuracy. Overall, the Backward Selection model chosen with Adjusted R-squared ended up having the lowest test MSE, making it the best for predicting the salaries. BIC models were simpler than the Adjusted R-squared ones but had slightly higher test errors.

PROBLEM SHEET 7

Comparison of least squares, ridge regression, LASSO and Elastic Net

We are still working with the Hitters dataset to predict Salary. First, make sure all missing values are dropped.

```
hitters_p7 = na.omit(Hitters)
```

1. Split the dataset randomly, keeping exactly 100 observations in the test set.

```
set.seed(42)
test_sample_idx = sample(1:nrow(hitters_p7), 100)

train_df = hitters_p7[-test_sample_idx, ]
test_df = hitters_p7[test_sample_idx, ]
```

2. Train the model using least squares and report the training and test MSE.

```
ls_fit = lm(Salary ~ ., data = train_df)

ls_train_mse = mean(resid(ls_fit)^2)
ls_predictions = predict(ls_fit, newdata = test_df)
ls_test_mse = mean((test_df$Salary - ls_predictions)^2)

cat("Least Squares Model:\nTraining MSE:", ls_train_mse, "\nTest MSE:",
    ls_test_mse, "\n\n")

## Least Squares Model:
## Training MSE: 91923.29
## Test MSE: 124714
```

3. Fit ridge regression models on the training set using a grid of λ values.

```
# Setting up the matrix for glmnet
x_train = model.matrix(Salary ~ ., data = train_df)[, -1]
y_train = train_df$Salary
```

```

x_test = model.matrix(Salary ~ ., data = test_df)[, -1]

# Ridge Regression uses alpha = 0. We use cross-validation to find the best
# lambda.
cv_ridge = cv.glmnet(x_train, y_train, alpha = 0)
best_lambda_ridge = cv_ridge$lambda.min

ridge_fit = glmnet(x_train, y_train, alpha = 0, lambda = best_lambda_ridge)

ridge_train_preds = predict(ridge_fit, s = best_lambda_ridge, newx = x_train)
ridge_test_preds = predict(ridge_fit, s = best_lambda_ridge, newx = x_test)

ridge_train_mse = mean((y_train - ridge_train_preds)^2)
ridge_test_mse = mean((test_df$Salary - ridge_test_preds)^2)

cat("Ridge Regression Model:\nBest Lambda:", best_lambda_ridge, "\nTraining
MSE:", ridge_train_mse, "\nTest MSE:", ridge_test_mse, "\n\n")

## Ridge Regression Model:
## Best Lambda: 29.83181
## Training MSE: 102403.4
## Test MSE: 104293.8

```